Text Flappy Bird with Monte Carlo Control and Sarsa(λ)

Mathys Bagnah

3MD3220 Reinforcement Learning | April 2026

Code and notebook: https://github.com/MathysBgh/Flappy_Bird_Reinforcement_Learning
Main notebook: `flappy_bird.ipynb`

Table 1: Core results on the training configuration.

Agent	Final eval	Best train	Last-100 mean
Random	–	2	0.24
Monte Carlo	49.00	49.00	9.69
Sarsa(λ)	49.00	49.00	36.24

Objective and Setup

The goal of this assignment is to compare two reinforcement learning agents on `TextFlappyBird-v0`: an on-policy first-visit Monte Carlo control agent and a tabular Sarsa(λ) agent. I selected the compact environment version rather than `TextFlappyBird-screen-v0`, because the state returned by `TextFlappyBird-v0` is directly the relative position to the next gap, (dx, dy), which is well suited to tabular value functions.

All main experiments use the same training configuration: height= 15, width= 20, and pipe-gap= 4. The Monte Carlo agent is trained for 2000 episodes with $\gamma = 0.99$ and an ε-greedy schedule from 0.35 to 0.05. Sarsa(λ) is trained for 4000 episodes with $\alpha = 0.10$, $\gamma = 0.99$, $\lambda = 0.70$, and an ε schedule from 0.30 to 0.01. The random baseline remains very weak, with mean score 0.24 and best score 2, which confirms that non-trivial performance must come from learning.

Learning Performance

Figure 1 highlights the main behavioural difference between the two algorithms. Both agents solve the training configuration and reach a final greedy evaluation score of **49.00**. However, Sarsa(λ) is clearly more sample-efficient late in training: its mean score over the last 100 training episodes is **36.24** versus **9.69** for Monte Carlo.

This difference is expected. Monte Carlo updates only after complete episodes, so learning is delayed and more irregular. Sarsa(λ), in contrast, updates online and uses eligibility traces, which propagate useful credit information faster through each episode. In practice, this produces smoother value estimates and a more stable rise in training performance.

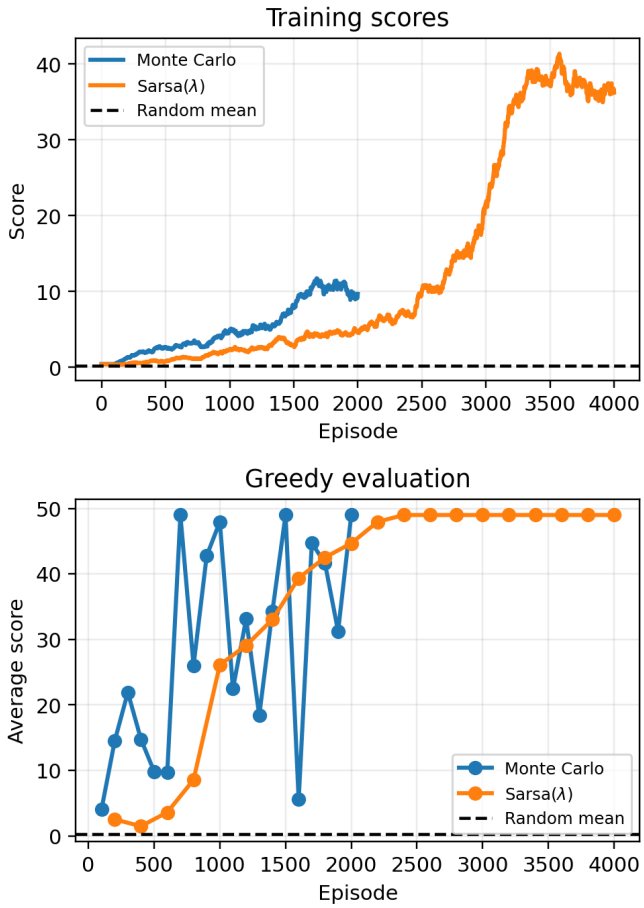

Figure 1: Training score evolution and greedy evaluation performance. Both agents solve the environment, while Sarsa(λ) reaches stronger late-training performance and a cleaner convergence profile.

State, Action-Value Functions, and Parameter Sensitivity

The learned value maps in Figure 2 are coherent: high-value regions are concentrated around safe and reachable states rather than appearing as unstructured noise. The Monte Carlo maps are meaningful but less smooth, whereas Sarsa(λ) produces more spatially regular value estimates. This matches the intuition from the learning curves.

Parameter sweeps were used to identify how robust each algorithm is to its key hyperparameters. For Monte Carlo, keeping ε_{end} too large hurts final performance because the policy keeps exploring too aggressively. For Sarsa(λ), the choice of λ has a visible effect on both performance and stability. In these runs, intermediate-to-high values of λ work best, but very large λ also increase variance.

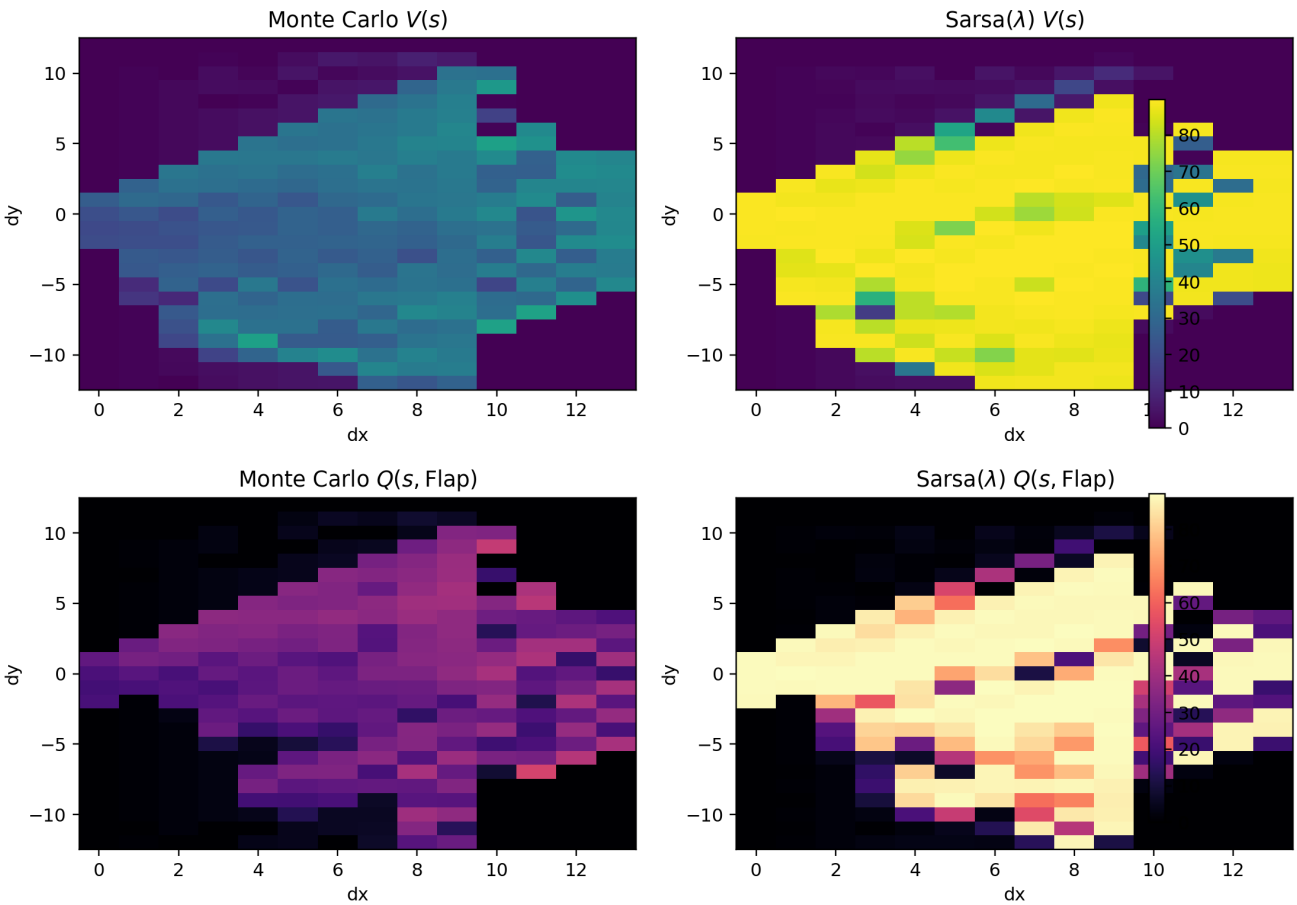

Figure 2: Learned state-value and action-value structure. The state-value maps $V(s)$ and the flap-specific action-value maps $Q(s, \text{Flap})$ reveal a structured safe region around reachable pipe-gap states. Sarsa(λ) produces smoother maps, which is consistent with its temporal-difference updates.

Environment Variants and Portability

The two TFB versions differ only in the observation returned to the agent. `TextFlappyBird-screen-v0` returns the full game screen as an integer matrix, which is richer but much more difficult to handle with tabular reinforcement learning. `TextFlappyBird-v0` returns only (dx, dy), which removes most of the representation burden and makes tabular Monte Carlo and tabular Sarsa(λ) practical. The limitation of the compact version is that it is less expressive; the limitation of the screen-based version is that it is not well matched to simple tabular value tables.

The same algorithmic ideas could be transferred to the original Flappy Bird environment, but not always in exactly the same implementation. If the state is compact and low-dimensional, the same tabular methods can be reused directly. If the original environment exposes raw images, the state space becomes too large for this implementation, and function approximation or deep reinforcement learning would usually be required instead.

Generalization to a New Configuration

To test transfer, the trained policies were evaluated on a new TFB configuration with height= 17, width= 22, and pipe-gap= 5. As expected, both agents lose performance because they were optimized for a specific geometry. Nevertheless, Sarsa(λ) transfers better: its average greedy score on the new configuration is **27.17** against **12.86** for Monte Carlo, while both agents remain perfect on the training configuration (49.00 and 49.00 respectively). This suggests that Sarsa(λ) learns a policy that is not only strong in-distribution, but also more robust under moderate environment changes.

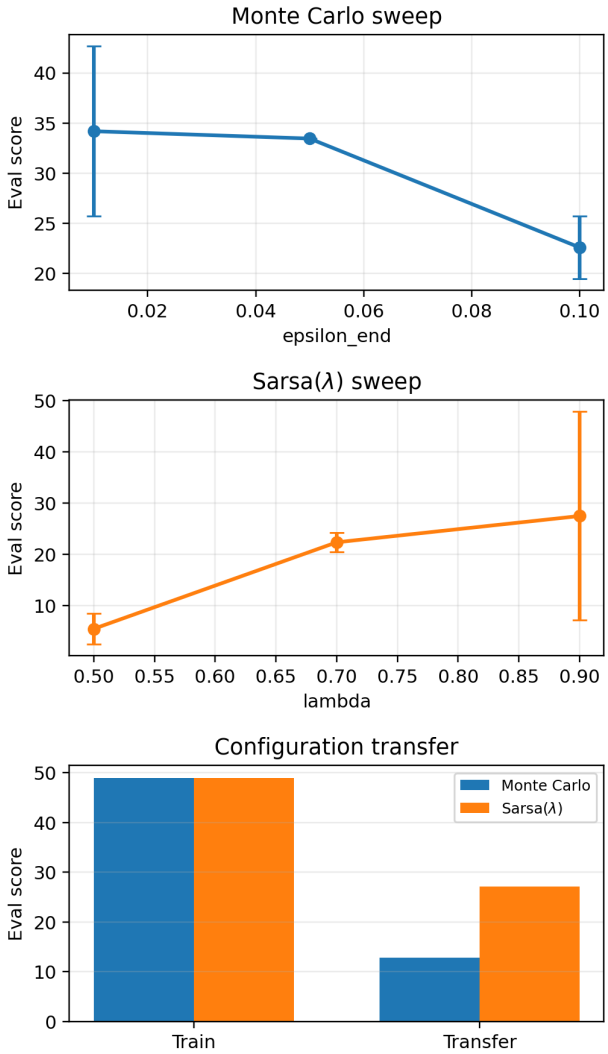

Figure 3: Hyperparameter sensitivity and transfer behaviour. Monte Carlo is sensitive to residual exploration, while Sarsa(λ) is sensitive to the eligibility-trace parameter and transfers better to the new level configuration.

Conclusion

Both implementations successfully solve `TextFlappyBird-v0`. Monte Carlo control eventually reaches optimal behaviour, but Sarsa(λ) learns faster, produces smoother value estimates, and generalizes better to the tested level change. Overall, the compact TFB environment is well suited to tabular reinforcement learning, while the screen-based version would likely require a stronger representation model to be competitive.

References

[1] Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*. 2nd edn. MIT Press (2018)

[2] Christodoulidis, S.: Text Flappy Bird Gym. https://gitlab-research.centralesupelec.fr/stergios.christodoulidis/text-flappy-bird-gym

[3] Talendar: Flappy Bird Gym. https://github.com/Talendar/flappy-bird-gym